Section 12.1: Global Traffic Management

> ***Part of a 3-file series*** *covering global systems design (Sections 12.1–12.3).*

Executive Overview

Global traffic management directs users to the nearest healthy region while maintaining compliance and optimizing cost. Engineers must balance latency, availability, and regulatory constraints when designing multi-region systems.

This section covers: GeoDNS vs. Anycast routing mechanics, global load balancing strategies, failure mode mitigations, and interview-ready design patterns.

Geographic DNS & Anycast Routing

What It Is

DNS and anycast provide different mechanisms for geographic routing. DNS operates at the **application layer** with TTL-based propagation; anycast operates at the **network layer** with BGP announcements. The choice between them comes down to propagation speed vs. routing control.

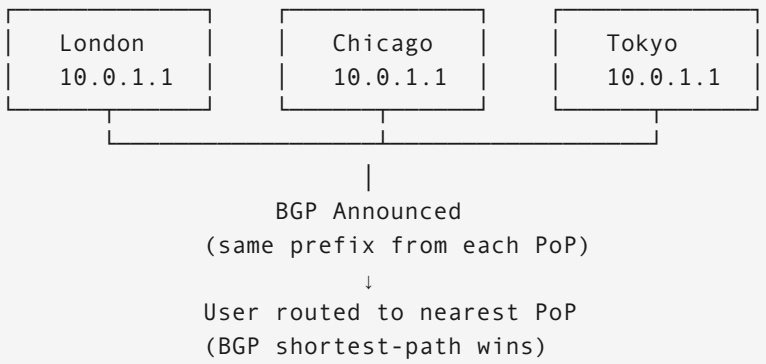
Core Mechanics

GeoDNS Architecture:

```
User DNS Query: api.example.com
        ↓
┌─────────────────────────────────────────────┐
│ GeoDNS Service (Cloudflare, AWS Route53)    │
│ - IP geolocation (MaxMind, IPinfo)          │
│ - Routing policy:                           │
│   - US    → us-east-1, us-west-2            │
│   - EU    → eu-west-1                       │
│   - APAC  → ap-southeast-1                  │
└─────────────────────────────────────────────┘
```

```
        ↓
Region-specific endpoint returned
```

Anycast Architecture:



Detailed Comparison: GeoDNS vs. Anycast

Dimension	GeoDNS	Anycast
Layer	Application (L7)	Network (L3)
Propagation speed	Minutes (TTL expiry)	Seconds (BGP convergence)
Failure detection	Health checks + TTL drain	BGP route withdrawal
Routing precision	Country / city-level	Nearest network hop
Compliance control	Full (can pin regions)	Difficult (BGP non-deterministic)
DDoS protection	Moderate	Strong (absorbs at edge)
Implementation complexity	Low	High (BGP, NOC required)

When to Use Each

Use GeoDNS when: - You have regulatory data-residency requirements (GDPR, PIPL) that mandate strict region pinning. - You need flexible traffic weighting (canary deployments, gradual rollouts). - Your organization doesn't operate its own BGP infrastructure. - Example: A banking API that must guarantee EU user data stays in `eu-west-1`.

Use Anycast when: - You need sub-second global failover (CDNs, DNS resolvers, DDoS scrubbing). - Your traffic is stateless and connection locality is not critical. - You operate globally distributed PoPs with BGP peering. - Example: Cloudflare's `1.1.1.1` DNS resolver — any Cloudflare PoP worldwide handles the query.

Use both together when: - GeoDNS directs users to a regional Anycast cluster; Anycast handles intra-region PoP selection. - Example: AWS Route53 Geolocation → Regional ALB → Anycast-routed edge nodes.

Global Load Balancing Strategies

What It Is

Global load balancers distribute traffic across regions based on health, latency, and capacity signals. Unlike single-region load balancing, global LB must account for cross-region replication lag, compliance constraints, and the cost of cross-region failover.

Core Mechanics

Health-Based Failover:

```
Health Checks every 30s:
  - HTTP 200 from /health endpoint
  - Latency < 100ms for probe request
  - Error rate < 1% over 5-minute window

Failure detected → Route traffic away from region
Recovery detected → Gradual traffic weight increase (10% → 50% → 100%)
```

Weighted Routing:

```
Normal:
  us-east-1  → 90%
  us-west-2  →  0% (standby)

Canary deploy:
  us-east-1  → 90% (stable)
  us-west-2  → 10% (new version)

Gradual cutover:
  us-east-1  → 50%
  us-west-2  → 50%
```

Performance-Based Routing (Latency-Aware):

```
RUM (Real User Monitoring) data:
  User in São Paulo → us-east-1: 180ms, sa-east-1: 40ms
  → Route to sa-east-1
```

```
Dynamic reweighting every 60s based on P95 latency
```

Detailed Comparison: Load Balancing Strategies

Strategy	Stability	Performance	Compliance-Safe	Best For
Health-based failover	High	Medium	Yes	Primary HA pattern
Weighted routing	High	Low	Yes	Canary, A/B, gradual rollout
Performance / latency	Low (oscillation risk)	High	Risky	Low-latency consumer apps
Geo-pinned	High	Medium	Excellent	Regulated industries

When to Use Each

Health-based failover: Default choice for any multi-region system. Simple, predictable, compliant. Use when SLA > availability optimization.

Weighted routing: Use for controlled feature rollouts and capacity management. Combine with feature flags for instant rollback without DNS changes.

Performance-based routing: Use for consumer applications where latency directly impacts revenue (e-commerce, gaming). Avoid if data residency applies — routing users to the fastest region may violate compliance.

Failure Modes & Mitigations

Split-Brain Scenarios

Problem: During a network partition, multiple regions believe they are primary and accept conflicting writes.

```
Normal:              Partition scenario:
us-east-1 (primary)  us-east-1 [thinks it's primary] ← writes from US users
    ↕ sync                  X  (partition)
eu-west-1 (replica)  eu-west-1 [thinks it's primary] ← writes from EU users
```

```
Result: Both regions diverge. On heal, conflicts exist.
```

Mitigations: - **Quorum writes**: Require acknowledgment from ≥ (N/2 + 1) regions before committing. With 3 regions, require 2 acknowledgments — a partitioned single region cannot form a majority and will reject writes. - **CRDTs (Conflict-free Replicated Data Types)**: Design data structures whose merge operations are always commutative. A distributed counter using CRDTs always merges correctly regardless of order. - **Vector clocks**: Track causal relationships across writes. On merge, resolve conflicts using happens-before ordering rather than wall-clock timestamps.

DNS Propagation Delays

Problem: With TTL = 300s (5 minutes), stale DNS responses continue routing users to a failed region for up to 5 minutes after failover.

Mitigations: - **Pre-reduce TTL**: Before any planned maintenance, reduce TTL to 30s 24 hours in advance (allowing the old TTL to expire). After the event, restore to 300s. - **Multiple DNS providers**: Use split-authority DNS (e.g., Route53 + Cloudflare) so that a DNS provider outage doesn't compound a region outage. - **Application-layer redirect**: Healthy regions serve `302` redirects for requests that arrive at the wrong region post-failover.

Region Outage Cascade

Problem: Single region failure overloads remaining regions with redirected traffic, causing secondary failures.

Mitigations: - **Active-active** (not primary/standby): All regions run at ~60% capacity, leaving headroom to absorb failover traffic. - **Cross-region failover target < 60s**. - **Graceful degradation**: Disable expensive features (recommendations, analytics) during overload; serve core functionality only. - **Load shedding**: Return `503` with `Retry-After` rather than cascading timeout failures.

Interview Design Problems

Problem 1: Global Banking API with Data Residency

Question: Design a global banking API that must comply with data residency regulations. EU users must be served from EU only. How do you route EU users while ensuring US users aren't affected by EU outages?

Solution:

Step 1 — Regulatory Architecture: - GeoDNS routes EU-sourced IPs → `eu-west-1` exclusively. All other IPs → `us-east-1` or `ap-southeast-1`. - TLS certificate pinning prevents application-layer cross-region calls. - EU data never replicates outside EU region (GDPR Article 44 compliance).

Step 2 — Failure Isolation:

```
EU region failure detected:
  1. Health checks fail for eu-west-1 (3 consecutive checks)
  2. GeoDNS: eu-west-1 record is NOT rerouted to US (compliance)
  3. Instead: GeoDNS returns EU-only failover record → eu-central-1 (Frankfurt
backup)
  4. If eu-central-1 also fails: EU users see "Service temporarily unavailable"
  5. US users: Entirely separate DNS path, unaffected
```

Step 3 — Key Insight: > Compliance trumps availability. The architecture must be willing to serve a "service unavailable" response to EU users rather than route them to a non-EU region. Build separate regional health-check trees; never share failover paths across compliance boundaries.

Trade-off acknowledged: EU users experience degraded availability during an EU-wide incident. This is acceptable — and legally required. Document this explicitly in your SLA.

Problem 2: Canary Deployment Across 3 Global Regions

Question: You are rolling out a new API version that changes the response schema. How do you use weighted routing to safely deploy across `us-east-1`, `eu-west-1`, and `ap-southeast-1` while limiting blast radius?

Solution:

Phase 1 — Single region canary (Day 1):

```
us-east-1  → v2: 5%,  v1: 95%
eu-west-1  → v1: 100%
ap-        → v1: 100%
```

Monitor: error rate, P99 latency, schema-validation errors for v2 traffic. Gate: 0% errors for 2 hours.

Phase 2 — Region-wide (Day 2):

```
us-east-1  → v2: 100%
eu-west-1  → v2: 5%,  v1: 95%
ap-        → v1: 100%
```

Phase 3 — Full global (Day 3+): All regions → v2: 100%.

Rollback trigger: If v2 error rate exceeds 1% at any phase, immediately shift weight back to 0% v2 via Route53 weighted records. New TTL-30s records drain in < 30 seconds.

Key design decision: Version routing at the load balancer layer (not DNS) for instant rollback — DNS TTL delay is unacceptable for incident response. Use path-based routing (`/v2/`) or header-based routing (`X-API-Version: 2`) at the ALB layer.

Problem 3: Anycast vs. GeoDNS for a Global CDN

Question: You are designing the traffic layer for a new CDN with 15 PoPs worldwide. When would you choose Anycast over GeoDNS for routing end users to the nearest PoP?

Solution:

Choose Anycast for this CDN. Reasons: 1. **Failover speed**: BGP route withdrawal reroutes users in seconds. DNS TTL causes 30–300s lag even with low TTL. 2. **DDoS absorption**: Anycast distributes volumetric attacks across all PoPs. GeoDNS concentrates attack traffic toward the resolved IP. 3. **Stateless workload**: CDN edge serves cached objects — no session affinity or data residency constraints. 4. **Operational simplicity at scale**: 15 PoPs announce the same prefix. Adding a 16th PoP requires only BGP configuration, not DNS updates per region.

Where GeoDNS is still needed: - Origin-tier routing (CDN → origin) may need GeoDNS to enforce data residency for origin pulls. - Customer-facing domain names (e.g., `cdn.customer.com`) still use CNAME → Anycast VIP.

Architecture:

```
User → Anycast VIP (nearest PoP by BGP)
     → Cache hit? Return object
     → Cache miss? GeoDNS to nearest compliant origin
```